Stage 1.4.2.5 — Choosing an OCR Approach and Running OCR

At this point, we already established an important distinction: **A scanned PDF page is essentially an image.**
So normal PDF text extractors (such as PyPDFLoader or PyMuPDFLoader) return little or no meaningful text.
Our goal now is:

TARGET PIPELINE

```
scanned PDF
    ↓
scanned_page.png
    ↓
OCR Engine
    ↓
recognized text
    ↓
LangChain Document
    ↓
later → chunking → embedding → vector DB → retrieval
```

For this learning stage, we start with **Tesseract OCR**.

1. Why Tesseract for this stage?

There are several OCR approaches available across the ecosystem:

OCR Approach	Type	Best Suited For
Tesseract	Open source	Learning OCR fundamentals, local processing
EasyOCR	Open source	Multilingual / general image OCR
PaddleOCR	Open source	Stronger document / layout OCR
Docling	Open source	Document understanding + structured extraction
Azure Doc Intelligence	Cloud	Enterprise document processing
Google Document AI	Cloud	Enterprise document understanding
AWS Textract	Cloud	Enterprise document extraction
OpenAI / Gemini Vision	Closed / API	Vision + reasoning + extraction

We shouldn't jump to Docling or cloud OCR yet. The purpose of Stage 1.4.2.5 is to understand the fundamental mechanism: *How do we turn pixels from a scanned page into text that can enter our RAG pipeline?* Tesseract is open-source and provides Windows installers via the UB Mannheim distribution.

2. Important: Tesseract has TWO parts

When we use import pytesseract, we have not installed the OCR engine itself. There are two distinct components:

TWO-PART ARCHITECTURE

```
Windows
│
├── Tesseract OCR Engine
│       ↓
│   actual OCR program (.exe binary)
│
└── Python
        ↓
    pytesseract
        ↓
    Python wrapper that communicates with Tesseract
```

Component 1: Tesseract executable engine.
Component 2: pytesseract Python wrapper.

3. OCR setup on your Windows 10 machine

Because you are using Windows 10, VS Code, Notebook, and uv, we'll keep the setup consistent with your existing environment.

Step 3.1 — Install Tesseract itself:
Download/install the Windows version from UB Mannheim. During installation, make sure English language data (eng) is selected.
Typical installation path: C:\Program Files\Tesseract-OCR\tesseract.exe

4. Verify Tesseract from PowerShell

After installation, close and reopen VS Code/PowerShell to refresh PATH environment variables. Run:

POWERSHELL

```
tesseract --version
```

Expected output:

OUTPUT

```
tesseract 5.x.x
 leptonica-...
 ...
```

Then check available language models:

POWERSHELL

```
tesseract --list-langs
```

OUTPUT

```
List of available languages in "..."
eng
osd
```

5. What if PowerShell says tesseract is not recognized?

This means Tesseract is installed but Windows cannot locate it in PATH. Test the absolute path first:

POWERSHELL

```
& "C:\Program Files\Tesseract-OCR\tesseract.exe" --version
```

If that works, add C:\Program Files\Tesseract-OCR to your Windows system/user PATH.

6. Now install the Python wrapper using UV

Go to your existing RAG project root and add the package:

POWERSHELL

```
cd <your-rag-learning-project>
uv add pytesseract
```

uv add updates pyproject.toml, lockfile, and virtual environment automatically.

DEPENDENCY SEPARATION

```
uv                  → manages Python dependencies (pytesseract)
Windows Installer   → installs Tesseract OCR binary engine
```

7. Verify pytesseract in your notebook

Create a new cell in your Stage 1.4.2.5 notebook and run:

PYTHON

```
import pytesseract

print(pytesseract.get_tesseract_version())
print(pytesseract.get_languages())
```

Output should reflect your installed engine version and ['eng', 'osd', ...].

8. Verify our actual scanned_page.png

Load the image produced from the previous experiment:

PYTHON

```
from PIL import Image

image = Image.open("scanned_page.png")

print(image.size)
print(image.mode)
```

Typical output: (1700, 2200), RGB.

9. Look at the scanned page before OCR

Inspect the visual frame:

PYTHON

```
display(image)
```

Remember: The PDF viewer sees structured text, but the OCR engine sees only raw pixel color arrays.

10. Run our first OCR

Execute the character recognition:

PYTHON

```
import pytesseract

ocr_text = pytesseract.image_to_string(
    image,
    lang="eng"
)

print(ocr_text)
```

FIRST OCR RUN

```
scanned_page.png → Image → pytesseract → Tesseract OCR → recognized text
```

11. Save the OCR result

Preserve the output text to disk:

PYTHON

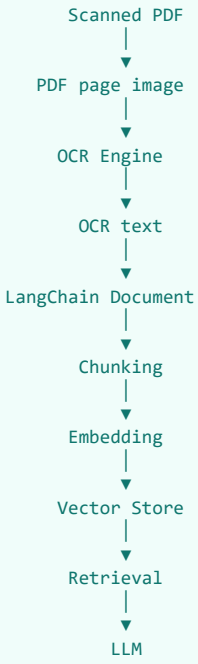
```
ocr_output_path = "scanned_page_ocr.txt"

with open(ocr_output_path, "w", encoding="utf-8") as f:
    f.write(ocr_text)

print(f"OCR output saved to: {ocr_output_path}")
```

This creates a clear visual transformation: scanned_page.png (IMAGE) → scanned_page_ocr.txt (TEXT).

12. Now connect OCR to LangChain

OCR is an ingestion capability. Here is how it completes the RAG flow:

INGESTION INTEGRATION



Without OCR: Scanned PDF → No text → No chunks → Retrieval fails.
With OCR: Scanned PDF → OCR Text → Chunks → Embeddings → Vector DB → Accurate Retrieval.

13. Create a LangChain Document

Convert the recognized text into a standard LangChain Document object:

PYTHON

```
from langchain_core.documents import Document

ocr_document = Document(
    page_content=ocr_text,
    metadata={
        "source": "scanned_page.png",
        "document_type": "scanned_pdf_page",
        "ocr": True,
        "ocr_engine": "tesseract",
        "language": "eng"
    }
)

print(ocr_document)
print(ocr_document.page_content)
```

14. Why metadata becomes especially important here

Preserving provenance (ocr: True) allows downstream debugging when tracking citation quality across multi-format corpora:

PROVENANCE TRACING

```
LLM answer
   ↓
retrieved chunk
   ↓
OCR-generated chunk (metadata: ocr=True, page=12)
   ↓
employee_handbook.pdf
```

15. One very important observation

OCR is prone to minor character hallucinations (e.g. Azure Event Hub → Azure Event Huh, or Data Explorer → Data Explorer). Because OCR errors degrade chunks, embeddings, and retrieval accuracy, production systems use preprocessing and quality evaluation.

16. Our first OCR experiment should be deliberately simple

Establish the baseline with raw image_to_string() first. Inspect word accuracy, line preservation, and table integrity before introducing advanced pre-processing filters (grayscale, noise removal, binarization, deskewing).

17. Also understand what Tesseract is NOT doing

Tesseract detects characters and words, but does not parse semantic relationships (e.g., mapping column keys to row values in complex tabular schemas). Advanced document understanding uses tools like PaddleOCR, Docling, Azure Document Intelligence, Google Document AI, or AWS Textract.

18. Your Stage 1.4.2.5 learning target

TARGET CHECKLIST

```
Stage 1.4.2.5
│
├── Understand OCR
├── Choose Tesseract
├── Install Tesseract on Windows
├── Install pytesseract using UV
├── Verify Tesseract
├── Load scanned_page.png
├── Run first OCR
├── Inspect OCR output
├── Save OCR text
└── Convert OCR output into LangChain Document
```

The upcoming progression:

ROADMAP

```
Stage 1.4.2.5: Basic OCR
      ↓
Stage 1.4.2.6: OCR preprocessing
      ↓
Stage 1.4.2.7: OCR quality evaluation
      ↓
Stage 1.4.2.8: OCR + PDF page pipeline
      ↓
Stage 1.4.2.9: OCR → LangChain Documents
```

One thing to do first

Before writing notebook code, verify your CLI environment:

POWERSHELL

```
tesseract --version
tesseract --list-langs
uv add pytesseract
```

Once these checks succeed, your environment is ready to execute OCR against scanned_page.png cell by cell.